STANDARD OPERATING PROCEDURE (SOP) | UNIVERSAL ARCHITECTURE BLUEPRINT v3.0

Headless WordPress + Astro Marketing Site

Evergreen Architecture, SEO, Pagination, Ad Tracking & Replication Guide

Doc ID: SOP-ENG-UNIVERSAL v3.0 | Stack: WPGraphQL + Gravity Forms + Astro + React 19 + Fontsource + Vercel Edge

Table of Contents

1. Executive Summary & System Architecture
2. Project Configuration & Placeholder Matrix
3. Phase 1: Headless WordPress Setup, Plugins & PHP Snippets
4. Phase 2: Astro Initialization, Fontsource & Edge Redirect Config
5. Phase 3: Folder Structure, Astro Image Pipeline & Zero-Inline-CSS Data
6. Phase 4: Dual-Mode WordPress API, Build Guards & Static Blog Pagination
7. Phase 5: Component Splits, Mobile Flyout Nav & Wave Geometry
8. Phase 6: Technical SEO, JSON-LD Schema Graphs & AI Snippet Blocks
9. Phase 7: Gravity Forms React Island, Ad Tracking & /thankyou/ Funnel
10. Phase 8: Vercel Deployment, WHM/cPanel DNS Cutover & Webhooks
11. Phase 9: New Site Replication & QA Checklist

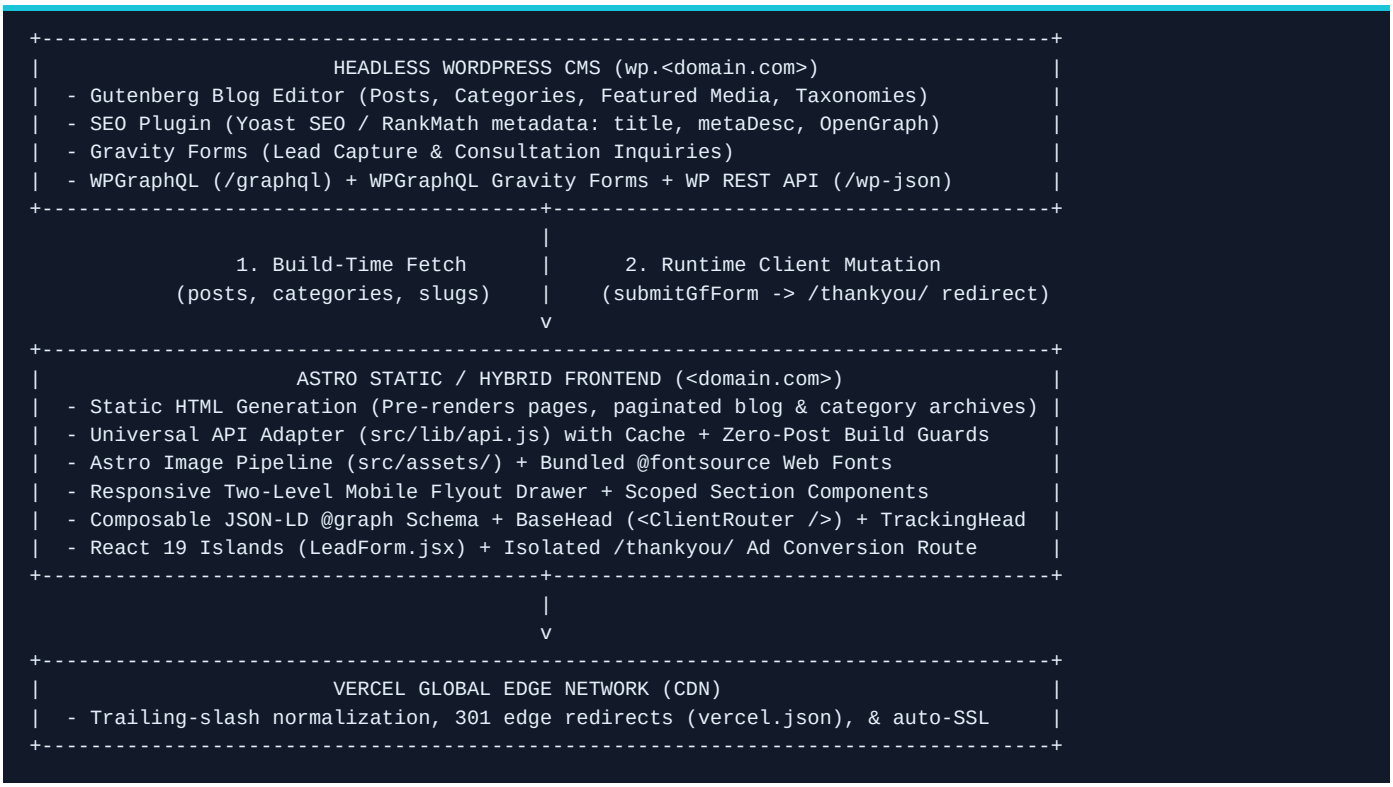
1. Executive Summary & System Architecture

1.1 Purpose & Scope

This Standard Operating Procedure (SOP) provides an evergreen, production-hardened engineering blueprint to build, migrate, and deploy high-converting marketing websites. By pairing Astro on the frontend with Headless WordPress on the backend, this architecture achieves:

- Sub-Second Core Web Vitals (CWV): 100% pre-rendered static HTML via Astro static site generation (SSG), self-hosted @fontsource typography, and build-time WebP/AVIF image optimization (astro:assets).
- Familiar Content Authoring: Non-technical marketing teams continue publishing blog posts, managing categories, and reviewing lead entries inside the standard WordPress dashboard.
- Enterprise-Grade SEO & Schema: Enforced trailing-slash canonical URLs, automated XML sitemaps, composable JSON-LD @graph structured data (Organization, LocalBusiness, Service, FAQPage, BlogPosting), static blog pagination, and category archives.
- Bulletproof Build Resilience: Fail-fast API Build Guards prevent silent content wipeouts if the WordPress origin is temporarily unreachable during a deployment.
- Isolated Conversion Tracking: Base Google Ads (gtag.js) and Meta Pixel scripts run globally via TrackingHead.astro, while lead conversion events fire exclusively on an isolated, noindex /thankyou/ route after a verified Gravity Forms submission.

1.2 End-to-End Architecture Topology



1.3 Core Technology Stack & Version Matrix

Layer	Package / Tool	Recommended Version	Architectural Purpose
Runtime	Node.js	>= 22.12.0	Modern LTS build environment & ES module execution
Framework	astro	^5.x or ^7.x	Static site generation (SSG), astro:assets, <ClientRouter />
UI Islands	react / react-dom	^19.x	Interactive client-side lead capture forms (@astrojs/react)
Typography	@fontsource/poppins / open-sans	^5.x	Self-hosted zero-latency WOFF2 font bundling
Animation	gsap	^3.15.x	ScrollTrigger parallax, counter animations, & floating graphics
Security	isomorphic-dompurify	^3.x	XSS sanitization for WordPress CMS HTML & confirmation messages
SEO	@astrojs/sitemap	^3.x	Automated sitemap-index.xml & sitemap-0.xml with route filtering
Icons	material-symbols	^0.40.x	Google Material Symbols Outlined font & ligature system
CMS Backend	WordPress + PHP	6.x / PHP 8.2+	Headless content management, taxonomies & Gravity Forms storage

2. Project Configuration & Placeholder Matrix

When replicating this architecture for a new client, brand, or agency project, replace the following template variables throughout your repositories and server configs:

Variable Token	Description	Example Target Value
<domain.com>	Production apex website domain	acmemarketing.com
<wp.domain.com>	Headless WordPress origin CMS domain	wp.acmemarketing.com
<brand>	Brand or client identifier (kebab-case)	acme-marketing
<brand-frontend>	Frontend Git repository and directory name	acme-marketing-frontend
<wp-server-ip>	IPv4 address of the WordPress / cPanel hosting server	192.0.2.45
<staging-url>	Staging or preview environment URL	https://staging2.acmemarketing.com
<form-id>	Target Gravity Form database ID	15 (or 1)
<google-ads-id>	Google Ads gtag.js Account ID & Conversion Label	AW-XXXXXXXXX / AW-XXXXXXXXX/YYYYY
<meta-pixel-id>	Meta (Facebook) Pixel ID	123456789012345
<company-phone>	Master client phone number	(555) 019-2834
<PREFIX>_VERCEL_DEPLOY_HOOK	PHP wp-config.php constant for deployment trigger	ACME_VERCEL_DEPLOY_HOOK

3. Phase 1: Headless WordPress Setup, Plugins & PHP Snippets

3.1 Subdomain Strategy

When decoupling an existing or new WordPress website:

1. Migration / Staging Phase: Keep the live website untouched while building on a staging subdomain (e.g., https://staging2.<domain.com>) pointed to Vercel via CNAME, fetching content from PUBLIC_WORDPRESS_API_URL.
2. Production Cutover Phase: Move or alias the WordPress installation to a dedicated origin subdomain (https://<wp.domain.com>) pointed to <wp-server-ip>. Point the apex domain (https://<domain.com>) and www to Vercel.

3.2 Required WordPress Plugins

Install and activate the following plugins in /wp-admin/plugins.php:

1. WPGraphQL (wp-graphql) -- Exposes /graphql for querying posts, categories, media attachments, and authors.
2. Gravity Forms (gravityforms) -- Commercial form manager for lead storage, admin notifications, and CRM webhooks.
3. WPGraphQL for Gravity Forms (wp-graphql-gravity-forms) -- Provides the submitGfForm mutation for direct browser submissions.
4. Yoast SEO (wordpress-seo) + WPGraphQL Yoast SEO Addon (add-wpgraphql-seo) -- Exposes seo { title metaDesc } on GraphQL post nodes and yoast_head_json on REST responses.

5. WP Webhooks (wp-webhooks) *(or custom save_post hook)* -- Triggers a Vercel rebuild when posts are published or updated.

3.3 Must-Use Plugin: wp-content/mu-plugins/headless-cms-config.php

Create a Must-Use (MU) plugin in WordPress so these rules remain permanently active regardless of theme changes:

```php
<?php
/**
 * Plugin Name: Universal Headless WordPress Configuration
 * Description: Optimizes WPGraphQL query limits, CORS headers, and Vercel build webhooks.
 * Version: 3.0.0
 */

// 1. Increase WPGraphQL max query limit for Astro single-fetch static builds
add_filter( 'graphql_connection_max_query_amount', function( $amount, $source, $args, $context, $info ) {
    return 500;
}, 10, 5 );

// 2. Allow CORS on WPGraphQL endpoint for frontend LeadForm submissions & staging
add_filter( 'graphql_response_headers_to_send', function( $headers ) {
    $allowed_origins = [
        'https://<domain.com>',
        'https://www.<domain.com>',
        'https://<staging-url>',
        'http://localhost:4321',
        'http://localhost:3000',
    ];

    $origin = isset( $_SERVER['HTTP_ORIGIN'] ) ? $_SERVER['HTTP_ORIGIN'] : '';
    if ( in_array( $origin, $allowed_origins, true ) ) {
        $headers['Access-Control-Allow-Origin']      = $origin;
        $headers['Access-Control-Allow-Methods']     = 'POST, GET, OPTIONS';
        $headers['Access-Control-Allow-Headers']     = 'Content-Type, Authorization';
        $headers['Access-Control-Allow-Credentials'] = 'true';
    }
    return $headers;
} );

// 3. Trigger Vercel Deploy Hook when a post is published or updated
add_action( 'save_post_post', function( $post_id, $post, $update ) {
    if ( wp_is_post_revision( $post_id ) || wp_is_post_autosave( $post_id ) ) {
        return;
    }
    if ( $post->post_status !== 'publish' ) {
        return;
    }

    $deploy_hook = defined( '<PREFIX>_VERCEL_DEPLOY_HOOK' ) ? constant( '<PREFIX>_VERCEL_DEPLOY_HOOK' ) : '';
    if ( ! empty( $deploy_hook ) ) {
        wp_remote_post( $deploy_hook, [
            'timeout'  => 5,
            'blocking' => false,
            'headers'  => [ 'Content-Type' => 'application/json' ],
            'body'     => json_encode( [
                'event'   => 'post_updated',
                'post_id' => $post_id,
                'slug'    => $post->post_name,
            ] ),
        ] );
    }
}, 10, 3 );
```

3.4 WAF Whitelisting & Gravity Forms Field Mapping

- Wordfence / Cloudflare WAF / ModSecurity: Whitelist /graphql for anonymous POST requests so client-side submitGfForm mutations are never blocked with HTTP 403.
- Gravity Forms Field Mapping (GF_FIELD_IDS):

Form Field Label	Field Type	Standard Field ID	GraphQL Mutation Input Key	Required
First Name	Single Line Text	12	value: String	Yes
Last Name	Single Line Text	13	value: String	Yes
Email Address	Email	5	emailValues: { value: String }	Yes
Phone Number	Phone (Standard)	6	value: String	Yes
Company / Organization	Single Line Text	4	value: String	No
Services of Interest	Text / Checkbox	7	value: String (comma-separated)	Yes
Project Details / Message	Paragraph Text	9	value: String	No

4. Phase 2: Astro Initialization, Fontsource & Edge Redirect Config

4.1 Bootstrap Command Sequence

```bash
# 1. Create a minimal Astro project
npm create astro@latest <brand-frontend> -- --template minimal
cd <brand-frontend>

# 2. Install official integrations, Fontsource typography, and core runtime libraries
npm install @astrojs/react @astrojs/sitemap @fontsource/open-sans @fontsource/poppins react react-dom gsap ...

# 3. Install React TypeScript definitions
npm install -D @types/react @types/react-dom
```

4.2 Universal package.json Specification

```json
{
  "name": "<brand-frontend>",
  "type": "module",
  "version": "1.0.0",
  "engines": {
    "node": ">=22.12.0"
  },
  "scripts": {
    "dev": "astro dev",
    "build": "astro build",
    "preview": "astro preview",
    "astro": "astro"
  },
  "dependencies": {
    "@astrojs/react": "^7.0.1",
    "@astrojs/sitemap": "^3.7.4",
    "@fontsource/open-sans": "^5.3.0",
    "@fontsource/poppins": "^5.3.0",
```

```
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "astro": "^7.3.8",
    "gsap": "^3.15.0",
    "isomorphic-dompurify": "^3.19.0",
    "material-symbols": "^0.40.2",
    "react": "^19.3.1",
    "react-dom": "^19.3.1"
  }
}
```

4.3 astro.config.mjs (Trailing Slashes, Image Domains & Sitemap Filtering)

Enforce trailingSlash: 'always', authorize remote WordPress media domains for Astro's <Image /> optimizer, and exclude alias/conversion routes from sitemap.xml:

```javascript
import { defineConfig } from 'astro/config';
import react from '@astrojs/react';
import sitemap from '@astrojs/sitemap';

export default defineConfig({
  site: 'https://<domain.com>',
  trailingSlash: 'always',
  image: {
    domains: ['wp.<domain.com>', '<domain.com>'],
  },
  integrations: [
    react(),
    sitemap({
      filter: (page) =>
        !page.includes('/paid-ads/') &&
        !page.includes('/social-media/') &&
        !page.includes('/sms-marketing/') &&
        !page.includes('/our-work/') &&
        !page.includes('/thankyou/'),
    }),
  ],
});
```

4.4 vercel.json (Edge 301 Redirects & Trailing Slash Enforcement)

Place vercel.json in the project root to handle legacy WordPress URL redirects at the CDN edge with zero client-side delay:

```json
{
  "trailingSlash": true,
  "redirects": [
    { "source": "/paid-ads/", "destination": "/services/pay-per-click-marketing/", "statusCode": 301 },
    { "source": "/social-media/", "destination": "/services/social-media-marketing/", "statusCode": 301 },
    { "source": "/sms-marketing/", "destination": "/services/sms-marketing-services/", "statusCode": 301 },
    { "source": "/our-work/:slug", "destination": "/work/:slug", "statusCode": 301 }
  ]
}
```

5. Phase 3: Folder Structure, Astro Image Pipeline & Zero-Inline-CSS Data

5.1 Complete Project Directory Tree

```text
<brand-frontend>/
+-- .env                            # Environment variables (PUBLIC_WORDPRESS_API_URL)
+-- astro.config.mjs                # Astro config (trailingSlash, image domains, sitemap filter)
+-- vercel.json                     # Edge 301 redirects & trailingSlash enforcement
+-- package.json                    # Dependencies & scripts
+-- public/                         # Unprocessed static files served at root "/"
|   +-- favicon.svg                 # Primary vector brand favicon
|   +-- brand-logos/                # Brand logos, favicons & OpenGraph default images
|   +-- main-graphics/              # 3D SVG centerpiece illustrations (1 per stage)
|   +-- web-assets/                 # Floating SVG badges, blobs & orbital satellites
|   `-- web-icons/                  # UI step badges & pillar SVG icons
`-- src/
    +-- assets/                     # Astro Image Pipeline (build-time WebP/AVIF optimization)
    |   +-- case-studies/           # High-res portfolio photography & project cards
    |   `-- graphics/               # Raster mockups & showcase imagery
    +-- config/
    |   `-- site.js                 # Global single source of truth (phone, email, meta defaults)
    +-- styles/
    |   +-- global.css              # CSS custom properties, reset, buttons, container grid
    |   `-- gutenberg.css           # Scoped typography & layout styles for WP blog content
    +-- lib/
    |   +-- api.js                  # Dual-mode REST/GraphQL client + Build Guards + Cache
    |   `-- animations.js           # Shared GSAP ScrollTrigger & floating animation helpers
    +-- layouts/
    |   +-- MainLayout.astro        # Global header, desktop mega-menu, MobileFlyoutMenu, JSON-LD
    |   `-- LandingLayout.astro     # Focused conversion layout for paid ad landing pages
    +-- components/
    |   +-- BaseHead.astro          # Canonical URL, OpenGraph, <ClientRouter />, JSON-LD script
    |   +-- TrackingHead.astro      # Isolated global Google Ads (gtag.js) & Meta Pixel base tags
    |   +-- MobileFlyoutMenu.astro  # Accessible 2-level slide-out mobile navigation drawer
    |   +-- ServiceHero.astro       # Master hero for /services/* pages
    |   +-- ServiceDefinition.astro # "What is [Service]?" callout box for AI/Featured Snippets
    |   +-- StartsSection.astro     # 2-column feature spotlight + animated SVG stat ring
    |   +-- StatsBand.astro         # Dark animated KPI number counter strip
    |   +-- ProcessSection.astro    # Interactive 4-step accordion + synced visual stage
    |   +-- PillarsSection.astro    # Numbered strategic pillars with parallax stage
    |   +-- DeliverablesGrid.astro  # 3-column deliverables grid with floating accents
    |   +-- ReadyBand.astro         # Dark #1a1a1a CTA surface extending behind consult waves
    |   +-- ConsultSection.astro    # Route-scoped wave ribbon + LeadForm section
    |   +-- CaseStudyHero.astro     # Master hero for /work/[slug]/ case studies
    |   +-- CaseStudyStats.astro    # KPI results strip for case studies
    |   +-- CaseStudyNarrative.astro  # Challenge / Solution / Results narrative layout
    |   +-- LightboxModal.astro     # Fullscreen image gallery lightbox
    |   +-- LeadForm.jsx            # React 19 Gravity Forms island (redirects to /thankyou/)
    |   +-- LeadForm.css            # Form layout, validation error states & pill checkboxes
    |   `-- landing/
    |       `-- FaqSection.astro    # Reusable FAQ accordion (paired with FAQPage schema)
    +-- data/
    |   +-- homeData.js             # Homepage hero slides, services grid & process steps
    |   +-- *Data.js                # Modular data models for each service & landing page
    |   `-- work/
    |       +-- caseStudiesRegistry.js # Central route map for case studies & legacy aliases
    |       +-- directoryData.js    # Portfolio hub filtering & Astro Image asset imports
    |       `-- *.js                # Individual case study data models
    `-- pages/
        +-- index.astro             # Homepage
        +-- about.astro             # About page
        +-- services.astro          # Services hub directory
```

```
        +-- services/*.astro          # Canonical service detail pages
        +-- work.astro                # Portfolio showcase directory
        +-- work/[slug].astro         # Canonical case study template (dynamic SEO titles)
        +-- thankyou.astro            # Isolated noindex conversion route (fires Google Ads event)
        `-- blog/
            +-- index.astro           # Knowledge Center main archive (Page 1)
            +-- [slug].astro          # Dynamic blog article template (DOMPurify + BlogPosting schema)
            +-- page/
            |   `-- [page].astro      # Static paginated blog archives (/blog/page/2/, etc.)
            `-- category/
                `-- [slug]/
                    `-- [...page].astro # Static paginated category archives (/blog/category/seo/)
```

5.2 Astro Image Pipeline (src/assets/) vs. Static public/ Assets

- Use src/assets/ (astro:assets <Image />) for all raster photography, portfolio cards, and case study mockups (.png, .jpg, .webp). Importing images as ES modules inside src/data/work/*.js allows Astro to verify file existence at build time, generate optimized WebP derivatives, and enforce width/height attributes to eliminate Cumulative Layout Shift (CLS).
- Use public/ for crisp vector SVGs (main-graphics/, web-assets/, web-icons/), favicons, and OpenGraph social share images that require stable root-relative URLs.

5.3 Zero Inline CSS Data Architecture

Data files (src/data//*.js) must never** contain inline style="..." strings:

- Data files define pure content: copy, links, image references, parallax depth numbers, and semantic Directional Placement tokens (placement: "top-left" | "top-right" | "bottom-left" | "bottom-right").
- Astro components map placement tokens to scoped CSS classes (.sat--top-left, etc.), keeping browser element.style exclusively reserved for runtime GSAP transforms (translate3d, rotate).

6. Phase 4: Dual-Mode WordPress API, Build Guards & Static Blog Pagination

6.1 Universal API Adapter with Zero-Wipeout Build Guards (src/lib/api.js)

src/lib/api.js supports both WPGraphQL (/graphql) and WP REST API (/wp-json/wp/v2) with an in-memory build cache (ensureFullPostsCache()). Crucially, it enforces Build Guards: if the WordPress API returns 0 posts during astro build, it throws an error and halts the build immediately so a temporary CMS outage never deploys an empty blog to production:

JAVASCRIPT

```javascript
// src/lib/api.js (Excerpt)
const API_URL = (
  import.meta.env?.PUBLIC_WORDPRESS_API_URL ||
  process.env?.PUBLIC_WORDPRESS_API_URL ||
  "https://<wp.domain.com>/wp-json/wp/v2"
).replace(/\/$/, "");

const isGraphQL = API_URL.endsWith("/graphql");

export async function getPosts(options = {}) {
  // When called with no arguments, return the full cached archive for pagination/category generation
  if (options.perPage === undefined && options.page === undefined) {
    return getAllPosts();
  }
  // ... fetch logic ...
  if (!Array.isArray(posts) || posts.length === 0) {
    throw new Error(
```

```
      `[Build Guard] WordPress API (${API_URL}) returned 0 posts. Failing build to prevent silent blog wipe...
    );
  }
  return posts;
}
```

6.2 Static Blog Pagination (src/pages/blog/page/[page].astro)

Generates crawlable, static paginated archive pages (/blog/page/2/, /blog/page/3/, etc.) with 12 posts per page:

```astro
---
import MainLayout from '../../../layouts/MainLayout.astro';
import { getPosts } from '../../../lib/api.js';

export async function getStaticPaths({ paginate }: any) {
  const posts = await getPosts();
  if (!posts || posts.length === 0) {
    throw new Error("No posts returned from API during pagination build.");
  }
  return paginate(posts, { pageSize: 12 });
}

const { page } = Astro.props;
---
<MainLayout
  title={`Blog - Page ${page.currentPage} | <Brand Name>`}
  description={`Read page ${page.currentPage} of our latest marketing insights and strategies.`}
>
  <!-- Render 12 post cards from page.data and Previous / Next pagination controls -->
</MainLayout>
```

6.3 Static Category Archives (src/pages/blog/category/[slug]/[...page].astro)

Automatically extracts every unique category across all WordPress posts and builds paginated category archives (/blog/category/<slug>/ and /blog/category/<slug>/2/):

```astro
---
import MainLayout from '../../../../layouts/MainLayout.astro';
import { getPosts } from '../../../../lib/api.js';

export async function getStaticPaths({ paginate }: any) {
  const posts = await getPosts();
  if (!posts || posts.length === 0) {
    throw new Error("No posts returned from API during category archive build.");
  }

  const categoryMap = new Map();
  posts.forEach((post: any) => {
    const categories = Array.isArray(post.categories) && post.categories.length > 0
      ? post.categories
      : (post._embedded?.['wp:term']?.[0] || []);

    categories.forEach((cat: any) => {
      if (cat && cat.slug && cat.slug !== "all") {
        if (!categoryMap.has(cat.slug)) {
          categoryMap.set(cat.slug, { name: cat.name, posts: [] });
        }
```

```
        categoryMap.get(cat.slug).posts.push(post);
      }
    });
  });

  return Array.from(categoryMap.entries()).flatMap(([slug, data]: [string, any]) =>
    paginate(data.posts, {
      params: { slug },
      props: { categoryName: data.name },
      pageSize: 12,
    })
  );
}
---
```

6.4 Sanitized Blog Post Rendering (src/pages/blog/[slug].astro)

All inbound WordPress HTML is scrubbed with isomorphic-dompurify while explicitly preserving responsive image attributes and embedded video iframes:

```
JAVASCRIPT
const sanitizedContent = post?.content
  ? DOMPurify.sanitize(post.content, {
      ADD_TAGS: ['iframe', 'figure', 'figcaption'],
      ADD_ATTR: ['target', 'allowfullscreen', 'frameborder', 'loading', 'decoding', 'sizes', 'srcset'],
    })
  : '';
```

7. Phase 5: Component Splits, Mobile Flyout Nav & Wave Geometry

7.1 Master Page Template Compositions

1. Service Detail Pages (src/pages/services/*.astro):
   - <ServiceHero /> -> <ServiceDefinition /> -> <StartsSection /> -> <StatsBand /> -> <PillarsSection /> -> <ProcessSection /> -> <DeliverablesGrid /> -> <ReadyBand /> -> <FaqSection /> *(optional)* -> <ConsultSection theme="dark" />
2. Case Study Pages (src/pages/work/[slug].astro):
   - Dynamic SEO Title (${study.client} Case Study: ${heroHighlight} | <Brand Name>)
   - <CaseStudyHero /> -> <CaseStudyStats /> -> <CaseStudyNarrative /> -> <LightboxModal /> -> <ConsultSection />

7.2 Responsive Two-Level Mobile Flyout Drawer (MobileFlyoutMenu.astro)

To prevent CSS backdrop-filter on .global-header from creating a containing block that clips fixed-position children on mobile viewports:

- Render <MobileFlyoutMenu /> as a direct sibling outside <header class="global-header"> inside MainLayout.astro with a top-level z-index: 9999.
- Two-Level Sliding Panel Architecture:
  - Level 1 (data-level="1"): Main site navigation links (Home, Services trigger button, Our Work, About Us, Blog) + direct phone link and Get Free Consult modal trigger (data-open-modal).
  - Level 2 (data-level="2"): Clicking Services transitions data-level="2" on #grmFlyoutDrawer, sliding Level 1 out to -30% and sliding the categorized Services submenu in from 100% -> 0% with a sticky Back to Main Menu bar.

7.3 Route-Scoped Wave & ReadyBand Integration

- On /services routes: ReadyBand.astro extends .ready-band__surface downward by the wave height (padding-bottom: clamp(55px, 8.5vw, 115px); margin-bottom: calc(-1 * clamp(55px, 8.5vw, 115px));). ConsultSection.astro uses a transparent wave container so the animated cyan waves rise directly over the dark patterned ReadyBand surface.
- On all other routes: ConsultSection.astro masks the top wave edge in white (var(--color-bg)), transitioning cleanly from light sections.

8. Phase 6: Technical SEO, JSON-LD Schema Graphs & AI Snippet Blocks

8.1 BaseHead.astro: Canonical Normalization, <ClientRouter /> & Staging Protection

src/components/BaseHead.astro centralizes all document <head> metadata:

1. Trailing-Slash Canonical Enforcement: Normalizes Astro.url.pathname to always end with / before constructing <link rel="canonical"> and <meta property="og:url">.
2. SPA View Transitions & GSAP Lifecycle: Renders <ClientRouter /> from astro:transitions so astro:page-load fires consistently across page navigations, re-initializing GSAP ScrollTrigger animations cleanly.
3. Staging & Private Route Protection: Outputs <meta name="robots" content="noindex,follow" /> whenever noindex={true} is passed (e.g., /thankyou/) or when deployed to a staging environment.

8.2 Composable JSON-LD @graph Structured Data

MainLayout.astro builds a base @graph containing the Organization (https://<domain.com>/#organization) and LocalBusiness (https://<domain.com>/#localbusiness) entities, and dynamically appends any page-level schema passed via pageSchema:

- Service Pages: Pass [serviceSchema, faqSchema] (@type: "Service" + @type: "FAQPage").
- Blog Posts: Pass blogSchema (@type: "BlogPosting" with headline, datePublished, dateModified, author, and publisher).
- Case Studies: Pass caseStudySchema (@type: "CreativeWork" / Article).

8.3 AI Overview & Featured Snippet Callouts (ServiceDefinition.astro)

Place <ServiceDefinition term="..." definition="..." /> directly beneath <ServiceHero /> on core service pages. This provides search engines and LLM answer engines with a concise, semantic <h2>What is {term}?</h2> and <p> definition block.

9. Phase 7: Gravity Forms React Island, Ad Tracking & /thankyou/ Funnel

9.1 Isolated Base Ad Tracking (src/components/TrackingHead.astro)

Render <TrackingHead /> inside the <head> of both MainLayout.astro and LandingLayout.astro. It loads only the global base trackers (Google Ads gtag.js config and Meta Pixel PageView), ensuring conversion events never fire prematurely on normal page views.

9.2 Gravity Forms React Island (src/components/LeadForm.jsx)

- Hydrated lazily via client:visible in ConsultSection.astro and on modal open in MainLayout.astro.
- Formats US phone numbers as (XXX) XXX-XXXX and validates required fields client-side.
- Submits entries via the WPGraphQL submitGfForm mutation (or REST fallback) and, upon verified success, redirects the user to /thankyou/:

```javascript
// Post-submission redirect inside src/components/LeadForm.jsx
if (json.is_valid !== false && typeof window !== 'undefined') {
  window.location.href = '/thankyou/';
}
```

9.3 Dedicated Conversion Route (src/pages/thankyou.astro)

/thankyou/ is rendered with noindex={true} and excluded from sitemap.xml. It fires the Google Ads conversion event exclusively when the thank-you page finishes loading:

```astro
---
import MainLayout from '../layouts/MainLayout.astro';
---
<MainLayout title="Thank You | <Brand Name>" noindex={true}>
  <script is:inline>
    window.addEventListener('load', function() {
      if (typeof gtag === 'function') {
        gtag('event', 'conversion', {'send_to': '<google-ads-id>/<conversion-label>'});
      }
    });
  </script>
  <section class="thank-you-section">
    <h1>Message Received</h1>
    <p>Thank you for reaching out. A strategist will be in touch shortly.</p>
    <a href="/">&larr; Return to Homepage</a>
  </section>
</MainLayout>
```

10. Phase 8: Vercel Deployment, WHM/cPanel DNS Cutover & Webhooks

10.1 Vercel Project Configuration

1. Import the GitHub repository into Vercel with Framework Preset Astro, Build Command astro build, Output Directory dist, and Node.js 22.x.
2. Add Environment Variables:
   - PUBLIC_WORDPRESS_API_URL = https://<wp.domain.com>/graphql *(or /wp-json/wp/v2)*
   - PUBLIC_WORDPRESS_GRAPHQL_URL = https://<wp.domain.com>/graphql

10.2 WHM / cPanel DNS Cutover Rules

When the domain's DNS is managed in WHM (DNS Zone Manager) or cPanel (Zone Editor):

- Golden Rule: Edit records strictly inside Zone Editor. Never add a Vercel staging or production domain under *cPanel -> Domains -> Create A New Domain*, which forces local Apache/LiteSpeed routing.
- Project-Specific Vercel Targets: Copy the exact CNAME (<hash>.vercel-dns-017.com.) and A record IP displayed in Vercel -> Settings -> Domains -> DNS Records.
- Email Continuity Check: If the client uses cPanel email, verify that mail.<domain.com>. has an A record pointing to <wp-server-ip> and the MX record points to mail.<domain.com>. before changing the root @ A record to Vercel.

Record Type	Host / Name	Target / Destination	TTL	Purpose
CNAME	staging2	<hash>.vercel-dns-xxx.com.	300	Staging preview subdomain (zero impact on live site)
A	wp	<wp-server-ip>	300	Hosts Headless WordPress CMS (wp.<domain.com>)
A	mail	<wp-server-ip>	300	Preserves cPanel mail routing (if using cPanel email)

A	@	<vercel-assigned-ip> *(or 76.76.21.21)*	300	Routes root domain (https://<domain.com>) to Vercel
CNAME	www	<hash>.vercel-dns-xxx.com.	300	Routes https://www.<domain.com> to Vercel with auto-SSL

11. Phase 9: New Site Replication & QA Checklist

[ ] 1. Brand & Global Configuration (src/config/site.js, astro.config.mjs, vercel.json):
- Replace <domain.com>, <wp.domain.com>, phone, email, and social links.
- Configure trailingSlash: 'always', image.domains, sitemap exclusions, and edge 301 redirects in vercel.json.

[ ] 2. Self-Hosted Typography & Design Tokens (MainLayout.astro & src/styles/global.css):
- Bundle @fontsource fonts and set brand CSS custom properties (--color-primary, --color-cyan, --color-dark).

[ ] 3. WordPress Backend & Build Guard Verification:
- Activate WPGraphQL, Gravity Forms, WPGraphQL for Gravity Forms, and headless-cms-config.php.
- Confirm src/lib/api.js Build Guards throw an error if 0 posts are returned.

[ ] 4. Content, Astro <Image /> Assets & Zero-Inline-CSS Compliance:
- Store raster portfolio/service photos in src/assets/ and vector SVGs in public/.
- Verify zero inline style="..." strings exist in src/data/**/*.js files.

[ ] 5. Technical SEO, Schema & Blog Pagination QA:
- Verify single <h1> per page, <ServiceDefinition /> blocks, and valid JSON-LD @graph output.
- Confirm /blog/page/[page]/ and /blog/category/[slug]/ static archives build cleanly.

[ ] 6. Mobile Flyout Menu & Conversion Funnel QA:
- Test the 2-level MobileFlyoutMenu.astro drawer across iOS and Android viewports.
- Submit a test entry in LeadForm.jsx, verify redirect to /thankyou/, and confirm the isolated Google Ads conversion event fires.